SPRING BOOT LIBRARY

Duplicate Check Annotation

Entity-level duplicate prevention for JPA

spring-boot-duplicate-checker-annotation 2.0.0 / 3.0.0 · Apache 2.0 · Java 17

Shivam Paliwal
Solutions Engineer · paliwalshivam93.vns@gmail.com

Technical documentation · October 2026

Contents

1. Overview 4

Why it exists . . . 4

At a glance . . . 4

Features . . . 4

2. Architecture 5

Components . . . 5

Request flow . . . 5

3. Installation 6

Choose a version . . . 6

Build and install . . . 6

Add the dependency . . . 6

Register the library . . . 7

4. Annotation reference 7

@CheckDuplicateEntity . . . 7

@UniqueField . . . 7

5. How rules are evaluated 8

Evaluation order for one rule . . . 8

Example: what the query looks like . . . 8

Rules of thumb . . . 9

6. Usage cookbook 9

Unique value . . . 9

Unique within a parent (nested path) . . . 10

Ignore soft-deleted or disabled rows . . . 10

Same name with the same set of locations . . . 10

Several independent rules . . . 10

Field-level style . . . 10

Switch checking off temporarily . . . 11

7. Error handling 11

DuplicateEntryException . . . 11

Default HTTP response . . . 11

Showing the error in the UI . . . 12

8. Operations 12

Performance . . . 12

Logging . . . 12

Testing . . . 13

9. Limitations and good practice 13

10. Version history 14

1. Overview

Duplicate Check Annotation is a Spring Boot library that stops duplicate records from being saved. You declare the uniqueness rules on the JPA entity, and the library enforces them automatically every time the entity is saved through a Spring Data repository.

Why it exists

Most business applications have rules such as "no two active applications with the same name in the same location" or "one assessment per department and period". Usually they are written by hand in each service as a repository lookup followed by an if-statement. That approach has problems:

- The same lookup-and-throw code is repeated in every service that creates or updates the entity.
- Any code path that forgets the check, such as an import job, a new endpoint or a bulk update screen, can still create duplicates.
- Composite rules involving foreign keys and many-to-many relations are long and easy to get wrong.
- Error messages and HTTP status codes end up inconsistent across modules.

With this library the rule lives on the entity, next to the data it protects. It is written once and applied to every save, whichever service triggers it.

At a glance

Item	Value
Artifact	`com.ascentbusiness:spring-boot-duplicate-checker-annotation`
Versions	2.0.0 for Spring Boot 2.x (javax) · 3.0.0 for Spring Boot 3.x (jakarta)
Java	17
Mechanism	Spring AOP `@Before` advice on repository `save(..)`, with JPA Criteria queries
Built on	Spring Boot, Spring Data JPA, Spring AOP, Hibernate 5.6 (v2), JHipster framework 7.8 (v2)
License	Apache License 2.0
Source	https://github.com/devnexus-sdp/duplicate_check_custom_annotation

Features

- **Single-field uniqueness:** for example, email must be unique.
- **Composite uniqueness** with `combinedWith`: name must be unique within a location, an RTO and an RPO.
- **Nested paths** through relations, such as `department.id`.
- **Many-to-many collections:** two records conflict only when they have exactly the same set of related IDs.

- **Conditional rules** with `onlyIfFieldEquals`, for example only among enabled or not-deleted records.
- **Update-safe:** the record being updated is excluded from its own check.
- **Clean API errors:** violations become `DuplicateEntryException`, returned as HTTP 409 Conflict.
- **Hibernate-proxy aware:** annotations on the real entity class are found even when a proxy is saved.

2. Architecture

Components

Class	Package	Role
`@CheckDuplicateEntity`	`ascentbusiness.annotation`	Marks an entity for checking and holds its rules
`@UniqueField`	`ascentbusiness.annotation`	One uniqueness rule
`EntityDuplicateCheckAspect`	`ascentbusiness.aspect`	Intercepts saves, evaluates the rules, runs the queries
`DuplicateEntryException`	`ascentbusiness.exception`	Thrown on a violation, carries the field, value and message
`GlobalExceptionHandler`	`ascentbusiness.exception`	Maps the exception to an HTTP 409 JSON response
`@CheckDuplicate`	`ascentbusiness.annotation`	Older method-level variant, deprecated and disabled
`DuplicateCheckAspect`	`ascentbusiness.aspect`	Aspect for the deprecated variant, not registered

Request flow

1. A controller or service calls `repository.save(entity)` on any `CrudRepository` or `JpaRepository`.
2. `EntityDuplicateCheckAspect` runs **before** the save. If the entity is a Hibernate proxy, the real class is used.
3. If the class has no `@CheckDuplicateEntity`, or has `enabled = false`, the save continues untouched.
4. Each `@UniqueField` rule is evaluated in order and becomes one JPA Criteria query on the same entity.
5. Many-to-many conditions are then checked in memory against the matching rows.
6. If any row is still left, `DuplicateEntryException` is thrown, the save never reaches the database, and the surrounding transaction rolls back.
7. `GlobalExceptionHandler` converts the exception into an HTTP 409 response with a JSON body.

```
Client --> Controller --> Service --> repository.save(entity)
                                            |
                            @Before  EntityDuplicateCheckAspect
                                            |
                          for each @UniqueField rule:
                            SELECT e FROM Entity e WHERE <rule predicates>
                            + in-memory check of collection fields
                                            |
                  no match: save continues     match: DuplicateEntryException
                                                        --> HTTP 409 Conflict
```

3. Installation

Choose a version

Your application	Library version	Namespace	Branch
Spring Boot 2.6+ (JHipster 7)	2.0.0	`javax.*`	`main`
Spring Boot 3.x (JHipster 8)	3.0.0	`jakarta.*`	`spring-boot-3.x`

Both lines use the same annotations, so moving an application from Spring Boot 2 to 3 needs only the version change.

Build and install

Install the library into your local Maven repository, or deploy it to your company repository such as Nexus or Artifactory:

```
git clone https://github.com/devnexus-sdp/duplicate_check_custom_annotation.git
cd duplicate_check_custom_annotation
git checkout main            # or: git checkout spring-boot-3.x
mvn clean install -DskipTests
```

Add the dependency

```
<dependency>
  <groupId>com.ascentbusiness</groupId>
  <artifactId>spring-boot-duplicate-checker-annotation</artifactId>
  <version>2.0.0</version>          <!-- 3.0.0 for Spring Boot 3 -->
</dependency>
```

The library depends on `spring-boot-starter-aop`, `spring-boot-starter-data-jpa` and `spring-boot-starter-web`, which most Spring Boot web applications already include.

Register the library

The aspect and the exception handler are ordinary Spring components in the `ascentbusiness` package. The library has no auto-configuration, so your application must scan that package:

```
@SpringBootApplication
@ComponentScan(basePackages = { "com.yourcompany.app", "ascentbusiness" })
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Alternatively, register only the aspect and write your own error handling (see chapter 7):

```
@Configuration
@Import(EntityDuplicateCheckAspect.class)
public class DuplicateCheckConfig { }
```

4. Annotation reference

@CheckDuplicateEntity

Placed on the entity class. Target: type. Retention: runtime.

Attribute	Type	Default	Description
`value`	`UniqueField[]`	`{}`	The rules to enforce. When empty, field-level `@UniqueField` annotations are used instead.
`enabled`	`boolean`	`true`	Set to false to switch checking off without deleting the rules.

@UniqueField

One uniqueness rule. Used inside `@CheckDuplicateEntity` (with `fieldName`), or directly on a field.

Attribute	Type	Default	Description
`fieldName`	`String`	`""`	Field to check, or a one-level path such as `rpo.id`. Required inside @CheckDuplicateEntity.
`message`	`String`	`"This value already exists"`	Error message. If left at the default, a message is generated instead (see chapter 7).
`combinedWith`	`String[]`	`{}`	More fields or paths that must also match. Collection fields may be written as `locations` or `locations.id`.
`onlyIfFieldEquals`	`String`	`""`	Name of a field that switches the rule on

Attribute	Type	Default	Description
`fieldValue`	`String`	`"true"`	Value that field must have. `"true"` and `"false"` become Boolean, whole numbers become Integer, anything else stays a String.
`excludeSelfOnUpdate`	`boolean`	`true`	Exclude the row with the same `id`, so updating a record does not conflict with itself
`groups`, `payload`			Bean Validation attributes, not used by the aspect

5. How rules are evaluated

This chapter describes exactly what the aspect does for each rule. It is useful when designing rules and when investigating why a save was or was not blocked.

Evaluation order for one rule

1. Read the value of `fieldName` from the entity being saved. **If it is null, the rule is skipped.**
2. If `onlyIfFieldEquals` is set, read that field. If it does not equal the converted `fieldValue`, the rule is skipped.
3. Build a query on the same entity type with these conditions combined by AND:
   • the main field equals the entity's value
   • every non-null, non-collection field in `combinedWith` equals the entity's value
   • the condition field equals `fieldValue`, if a condition is set
   • `id` is not the entity's own id, when `excludeSelfOnUpdate` is true and the entity already has an id
4. Run the query. If `combinedWith` includes collections, keep only rows whose collection has **exactly the same set of IDs**.
5. If any row remains, throw `DuplicateEntryException`. The first violated rule stops the save.

Example: what the query looks like

For this rule on `SaApplicationSoftware`:

```
@UniqueField(fieldName = "name",
             combinedWith = { "locations.id", "applicationRto.id", "rpo.id" },
             onlyIfFieldEquals = "isEnabled", fieldValue = "true")
```

Saving an enabled record named "SAP" with RTO 4, RPO 9 and locations {1, 3} runs the equivalent of:

```
SELECT s FROM SaApplicationSoftware s
 WHERE s.name = 'SAP'
   AND s.applicationRto.id = 4
   AND s.rpo.id = 9
   AND s.isEnabled = true
   AND s.id <> :currentId          -- only when updating
```

Then, among the rows found, only those whose locations are exactly {1, 3} count as duplicates. A record with locations {1} or {1, 3, 5} is **not** a duplicate.

Rules of thumb

Situation	Behaviour
Main field is null	Rule skipped, so nulls never conflict
A `combinedWith` field is null	That condition is left out of the query, so the rule becomes broader
Condition field does not match	Rule skipped for this save
Collection in `combinedWith`	Compared as a set of `Long` IDs, order does not matter
Path deeper than one level, such as `a.b.c`	Not supported: an error is logged and the rule is skipped
String comparison	Plain equality, so case sensitivity follows your database collation
Several rules on one entity	Checked in declaration order, and the first violation wins

6. Usage cookbook

Unique value

```
@Entity
@CheckDuplicateEntity({
    @UniqueField(fieldName = "email",
                 message = "A customer with this email already exists")
})
public class Customer {
    @Id @GeneratedValue private Long id;
    private String email;
}
```

Unique within a parent (nested path)

```
@Entity
@CheckDuplicateEntity({
    @UniqueField(fieldName = "code", combinedWith = { "department.id" },
                 message = "This code is already used in the department")
})
public class CostCentre {
    @Id private Long id;
    private String code;
    @ManyToOne private Department department;
}
```

Ignore soft-deleted or disabled rows

```
@CheckDuplicateEntity({
    @UniqueField(fieldName = "name",
                 onlyIfFieldEquals = "isDeleted", fieldValue = "false",
                 message = "An active plan with this name already exists")
})
public class ContinuityPlan { ... private Boolean isDeleted; }
```

The check runs only when the saved record has `isDeleted = false`, and compares it only with other records that also have `isDeleted = false`. Deleted records never block new ones.

Same name with the same set of locations

```
@CheckDuplicateEntity({
    @UniqueField(fieldName = "name", combinedWith = { "locations" })
})
public class Site { ... @ManyToMany private Set<OrgLocation> locations; }
```

Several independent rules

```
@CheckDuplicateEntity({
    @UniqueField(fieldName = "email",    message = "Email already registered"),
    @UniqueField(fieldName = "employeeNo", message = "Employee number already in use")
})
public class Employee { ... }
```

Field-level style

Leave the class annotation empty and annotate the fields. This only suits simple single-field rules:

```
@Entity
@CheckDuplicateEntity
public class Product {
    @Id private Long id;
    @UniqueField(message = "SKU already exists") private String sku;
}
```

Switch checking off temporarily

```
@CheckDuplicateEntity(enabled = false, value = { @UniqueField(fieldName = "email") })
```

7. Error handling

DuplicateEntryException

Member	Description
`getMessage()`	The rule's message, or a generated one
`getField()`	Field or path that was duplicated, for example `name` or `rpo.id`
`getValue()`	The duplicated value

If a rule keeps the default message, the text is generated as `Duplicate value 'SAP' found for field 'name'`.

Default HTTP response

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "timestamp": "2026-10-29T10:15:30.512",
  "status": 409,
  "error": "Duplicate Entry",
  "message": "Duplicate record found",
  "field": "name",
  "value": "SAP"
}
```

> **Check the bundled handler before scanning it.** `GlobalExceptionHandler` also has a catch-all handler for **every** `Exception`, returning HTTP 500. In an application with its own error handling, such as JHipster's `ExceptionTranslator` with Problem responses, this can take over or conflict with existing error responses. In that case, leave the handler out and map the exception yourself:

```
@ComponentScan(
    basePackages = "ascentbusiness",
    excludeFilters = @ComponentScan.Filter(
        type = FilterType.ASSIGNABLE_TYPE,
        classes = GlobalExceptionHandler.class))

@RestControllerAdvice
public class DuplicateEntryAdvice {
    @ExceptionHandler(DuplicateEntryException.class)
    public ResponseEntity<ProblemDetail> handle(DuplicateEntryException ex) {
        ProblemDetail pd =
            ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
        pd.setProperty("field", ex.getField());
        return ResponseEntity.status(HttpStatus.CONFLICT).body(pd);
    }
}
```

`ProblemDetail` needs Spring 6 / Spring Boot 3. On Spring Boot 2, return a `Map` or your own error DTO.

Showing the error in the UI

```
this.http.post('/api/customers', customer).subscribe({
  error: (err: HttpErrorResponse) => {
    if (err.status === 409) {
      this.form.get(err.error.field)?.setErrors({ duplicate: err.error.message });
    }
  }
});
```

8. Operations

Performance

- Each rule costs one SELECT per save. Index the columns you check, using the same column combination as the rule.
- Collection rules load the matching rows and compare their collections in memory. Keep the other conditions selective, such as name plus foreign keys, so only a few rows are loaded.
- Bulk imports that call `save` row by row run one query per rule per row. Plan batch sizes accordingly.

Logging

The aspect logs at INFO level on every save, which is very verbose in production. Lower the level for the library:

```
# application.yml
logging:
  level:
    ascentbusiness: WARN      # keeps "Duplicate entry detected" warnings and errors
```

Testing

```
@SpringBootTest
@Transactional
class CustomerDuplicateTest {
    @Autowired CustomerRepository repository;

    @Test
    void secondCustomerWithSameEmailIsRejected() {
        repository.save(new Customer("a@acme.com"));
        assertThrows(DuplicateEntryException.class,
                     () -> repository.save(new Customer("a@acme.com")));
    }
}
```

9. Limitations and good practice

Limitation	Recommendation
Only `save(..)` is intercepted, not `saveAll`, `saveAndFlush`, JPQL updates or native SQL	Route writes for protected entities through `save`, or add your own checks on the other paths
The check and the insert are separate steps, so two concurrent requests can both pass	Keep a database unique constraint or index as the final guarantee where possible
Fail-open: an error inside a rule, such as a misspelled field name, is logged and the save is allowed	Cover each rule with a test like the one in chapter 8
Fields are looked up on the entity class itself, so fields inherited from a superclass are not found	Declare checked fields on the entity class
The entity must have a field named `id` for update exclusion	Use `id` as the identifier name, as JHipster does
`fieldValue` converts numbers to Integer, so Long or enum condition fields never match	Use Boolean, Integer or String fields for conditions
Nested paths support one level only	Point at the foreign key, for example `department.id`

10. Version history

Version	Date	Spring Boot	Changes
3.0.0	27 Jul 2026	3.0+	Jakarta EE namespace, JHipster 8 compatibility
2.0.0	27 Jul 2026	2.6.6+	Nested paths, @ManyToMany collections, both locations and locations.id syntax, update exclusion
1.0.0	22 May 2026	2.6.6+	First release: single-field and composite rules

Licensed under the Apache License 2.0. Source:
https://github.com/devnexus-sdp/duplicate_check_custom_annotation